

How we faked it

Async event handlers as container morphisms:
composing shapes forwards while responses flow back

September 1, 2026

Abstract

The fake is not a type-level trick with a network attached. **The fake is the async event handler.** A container’s inhabitant $(p : P) \rightarrow R p$ is a handler woken by a prompt; a morphism between containers sends shapes *forward* and brings responses *back*; and **await** is what allows both directions to be written as one straight-line function, in a server that is simultaneously a client. This note is about that, and about what the type system does and does not contribute to it.

1 A container is an event handler

§3 of Ghani’s note observes that event-driven programming “has a feature that turns out to be exactly a dependent type”: which responses are appropriate depends on which event arrived. A container (P, R) is that feature written down — P the prompts, $R p$ the positions, the responses appropriate to p — and an inhabitant of the interface is the dependent function $(p : P) \rightarrow R p$.

In this project that inhabitant is not a metaphor. It is the handler:

```
serveContainer({
  name: "agri", port: 8802,
  parse: (u: unknown) => AgPrompt | null,      // the prompt arrived as bytes
  answer: (p: AgPrompt, depth) => Promise<...>, // <-- THIS is (p : P) -> R p
});
```

answer is woken by a socket, receives one prompt, and owes exactly the response type that prompt indexes. Five such handlers are running while the game is played.

2 Two directions

A morphism of containers $(P, R) \rightarrow (Q, T)$ is a pair:

$$u : P \rightarrow Q \qquad f : (p : P) \rightarrow T (u p) \rightarrow R p$$

u turns my prompt into one for the level below — **shapes travel forwards**. f turns their response into mine — **responses travel back**. Control descends; data ascends. In a game about a command economy that is not a coincidence: Gosplan does not reap anything. It converts “feed the towns” into “Narkomzem: harvest”, and converts the reply back into a number for its own ledger.

The tower prints both directions as it runs. One **Sow**, across four processes:

```
:8801 |- gosplan   Sow order={"procurement":"firm","labour":{...
:8801 |   |- gosplan ((agri (x) smelter) (x) transport)
:8802 |     |- agri   Harvest workforce={"fields":97,...} tractors=0 year=1928
:8802 |     -| agri   {grain, grade, perWorker, withoutTractors}
:8803 |     |- industry Smelt workers=16 millCapacity=8 year=1928
:8803 |     -| industry {steel, idleCapacity}
:8804 |     |- transport Lay workers=7 steel=10 year=1928
:8804 |     -| transport {added, steelUsed}
:8801 -| gosplan   {kind, year, harvest, steel, ...}
```

`| -` is a shape going forward, `-|` a response coming back. Every indent is a morphism.

3 await composes the two directions

Here is the whole of a morphism, in code:

```
const reply = await lower.run(u(prompt)); // shapes forward
return f(prompt, reply);                 // responses back
```

Both directions, one function body, read top to bottom.

`await` is what makes that possible. A handler is woken by a prompt, and before it can answer it usually needs answers from below. At the `await` it *suspends*: it hands control back to the runtime, which is then free to wake other handlers, and it resumes at that same point with the reply in hand. So u is applied on the line going down, f on the line coming back, and the suspension between them is invisible in the text of the function.

Why this closes the algebra. A morphism written this way is an ordinary `async` function from a prompt to its response — which is precisely what a container’s inhabitant is. So composing two morphisms is just calling one inside the other, and the composite is again an `async` function from a prompt to its response. **The morphisms are closed under composition in the code, in the same way they are closed under composition in the theory**, and a five-level tower is written as five ordinary functions calling one another.

And it is why a server can also be a client. Gosplan is woken by a socket and, while answering, delegates to four commissariats. Because each of those delegations suspends rather than occupies the runtime, Gosplan remains a listening server throughout — one handler, mid-answer, with four prompts outstanding beneath it.

4 The combinators, in both directions

Each combinator is a way of composing the forward flow, and each has a matching rule for the backward one.

	shapes forward	responses back
⊠	a prompt to each side, dispatched together	a reply from <i>each</i> ; the position is a pair
×	a prompt to each side	a reply from <i>exactly one</i>
+	a prompt to one side	that side’s position, unchanged
◁	the second prompt is <i>built from the first reply</i>	a pair of replies

⊠ — forward together

```
const [left, right] = await Promise.all([a.run(s.left, d), b.run(s.right, d)]);
return { left, right };
```

Both shapes existed before the call, so neither can depend on the other’s reply — independence is enforced by the shape type `{left, right}` itself: both prompts had to exist before the call. The `Promise.all` sends them together, so the tensor costs one round trip rather than two.

◁ — where the two directions interleave

This is the one that needs the hinge, and the reason the sequential composition is the hard combinator:

```
const first = await a.run(s.first, depth); // forward, then BACK
const second = await b.run(s.next(first), depth); // the reply BECOMES the
                                                    // next shape: forward again
```

`s.next(first)` is the whole content of `◁`. A response flowing back turns around and becomes a shape flowing forward. The composite’s shape is therefore not a pair of prompts but a prompt *together with a function* from its replies to the next prompt:

```
step<SA extends Shape<A>, SB extends Shape<B>>(  
  first: SA,  
  next: (r: PosAt<A, SA>) => SB,          // SA inferred from 'first', so 'next'  
)                                          // is typed at THAT fibre alone
```

In the game: what you may *sell* depends on what the railway actually *moved*, so the trade delegation is asked its question only once the haul has answered.

5 The tower makes the chain real

u is not a function call. It is a CLI tool:

```
delegate.ts <port> <json-prompt> <depth>      # stdout is the reply
```

It knows nothing about any container — port, JSON, depth, and it carries the text across. That ignorance is the point: *u* is a map of shapes, and shapes are what survive a wire. A CLI tool calls a CLI tool, whose server calls another, and the responses compose back up the same chain; the `depth` header is what indents the trace above.

What depth is. A count of container boundaries crossed. The player's client sends zero; each handler reads it, traces at that indent, and delegates at `depth + 1`:

```
const depth = Number(req.headers.get("x-tower-depth") ?? "0");  
trace(depth, "down", opts.name, describe(prompt));  
const reply = await opts.answer(prompt, depth + 1);
```

It rides in an HTTP header rather than in the prompt, and that is a typing decision rather than a convention. The prompt is the container's *shape*, so putting a depth in it would grow every shape type in the game by a field — `{tag:"Harvest", workforce, tractors, year, depth}` — and *R* would then be indexed over the plumbing. The header is the channel for things that must cross a boundary without joining the container, which is the same instinct that keeps `Promise` out of it: both describe how an answer reaches you, not what the answer is.

It earns its place because each server logs to its own file. Five flat streams carry no record of what nested inside what; the counter is the only thing tying `agri`'s `Harvest` to the `Sow` that caused it, and it is what lets them be read back as one tree. Nothing in the game consults it — set it wrongly and only the indentation suffers.

Why processes rather than function calls. Each commissariat holds a hidden efficiency and a bias — how much it inflates a shortfall — and the player plans against the reports, not the truth. Inside one process that privacy is a convention, one stray import from collapsing. Across five processes with only JSON between them the operating system enforces it: there is no reference to reach through. The game's subject is an information asymmetry, and the wire is what makes it a fact rather than a promise, at about a fifth of a second per hop.

6 What the type system contributes

The handler is the fake; the types are what hold it to its word.

The container as its graph

A container is presented as the union of its fibres, which over a *finite* index is exactly its Σ :

$$\Sigma(s : S). B s = \bigcup_{s \in S} \{s\} \times B s$$

```
interface Fib<S, P> { readonly shape: S; readonly pos: P }  
type PosAt<C extends Cont, S> =  
  C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) : never;
```

`Fib` is never constructed; it is taken apart by `infer`. The handler’s signature then reads `run<S extends Shape<C>>(shape: S): Promise<PosAt<C, S>>` — ask for a harvest, be handed a harvest report, uncast.

The finite index is what makes it exact

That equality is the enabling condition, and it is worth being precise about why it holds. The general definition of a container, as it appears in Agda,

```
record Container :   Shape : Set,      Pos : Shape → Set
```

asks for `Pos` to be a function from *values* to *types* — a genuine dependent function, which is why containers are a dependently typed idea and why the canonical example is the list functor, `Shape = ℕ` with `Pos n = Fin n`.

Every container in this project has *finitely* many prompts, because a shape type is a tagged union. Over a finite index the graph presentation is not an approximation of Σ but an equality, and a conditional type distributing over a union computes it exactly. That is the whole reason the encoding is sound: not that TypeScript is cleverer than it looks, but that the containers being encoded are small enough to write down fibre by fibre.

The same condition governs the same construction elsewhere. In Haskell the finite case is a GADT indexed by its response type — `Reconcile :: (Int,Int) -> Request ReconReport` — and `answer :: Request r -> r` is the dependent function, checked branch by branch with no casts at all.

The bill

Ten casts, two per combinator, because every `run` body deconstructs a shape it knows through a conditional type and re-asserts the result. All ten sit inside `lib/algebra.ts`; none is in the game, where no call site ever asserts the type of a position.

Two pieces of machinery earn their keep quietly. `PosAt` tests `S extends S1` — the fibre a held shape *lies in*, since a shape in hand is usually narrower than its fibre’s declared type. And `Runner` carries a phantom `readonly fibres?: C`, never assigned, which gives `C` an inferable position so that `seqC(a, b)` recovers its arguments.

Summary

The fake	an <code>async</code> event handler: $(p : P) \rightarrow Rp$, woken by a socket
What it buys	shapes compose <i>forwards</i> through u and responses compose <i>backwards</i> through f , in one straight-line function
Why it composes	a morphism is an ordinary <code>async</code> function from prompt to response, so composing two is calling one inside the other
Why it scales	suspension rather than occupation, so a handler can be mid-answer with four prompts outstanding and still be listening
The rich combinator	$\triangleleft$, where a response turns around and becomes the next shape
What types add	the response type follows from the prompt, uncast, at ten casts confined to one library
What makes it exact	finitely many prompts per container, so the graph presentation of Σ is an equality